Mini Bootcamp Data Engineering Project

By Dibimbing.id

DIGITAL SKILL FAIR 44.0: FACULTY OF DATA ENGINEERING

Project Name: End-to-end Job Market Trends Pipeline

by Fauzan Dharmawan

⚠ **Disclaimer**

This project is created solely for educational and learning purposes

All data collected and analyzed in this project are **limited to publicly accessible information** and comply with website's robots.txt policy.

Problem Summary

The job market changes rapidly, making it difficult to clearly understand current hiring trends, company demand, and employment patterns in real time. Raw job posting data—such as LinkedIn listings—often comes in semi-structured formats and requires cleaning, transformation, and aggregation before meaningful insights can be generated. The goal of this end-to-end Job Market Trends Pipeline is to process and standardize job posting data so that we can analyze recent hiring activity and identify actionable trends over the last 30 days.

Key Business Questions to Answer

- *What are the job trends in the last 30 days?*

Identify which roles, functions, industries, or locations have experienced increased posting activity to understand demand shifts in the market.

- *Which companies are hiring the most in the last 30 days?*

Determine which organizations have posted the highest number of job listings, indicating aggressive hiring or expansion.

- *What is the employment type trend?*

Analyze the distribution and growth of employment types (e.g., Full-time, Contract, Internship, Remote) to understand workforce structure changes and hiring preferences.

Scope of Project

This project focuses on **building an end-to-end data pipeline** that ingests publicly available job posting data, processes and standardizes semi-structured information, and transforms it into structured datasets for analytical use. The scope includes data collection (within **robots.txt** compliance), data cleaning and transformation, storage in a structured format, and aggregation of job postings from the **last 30 days**. The project will generate trend-based insights related to job roles, hiring companies, locations, industries, and employment types. It includes automated data processing workflows and visualization/dashboarding for trend analysis but does not cover predictive modeling, salary analysis, or private/non-public data sources. The primary objective is to demonstrate data engineering, ETL Concept, and analytical capabilities through a simple pipeline architecture.

1. Tools

Python Programming

SQL

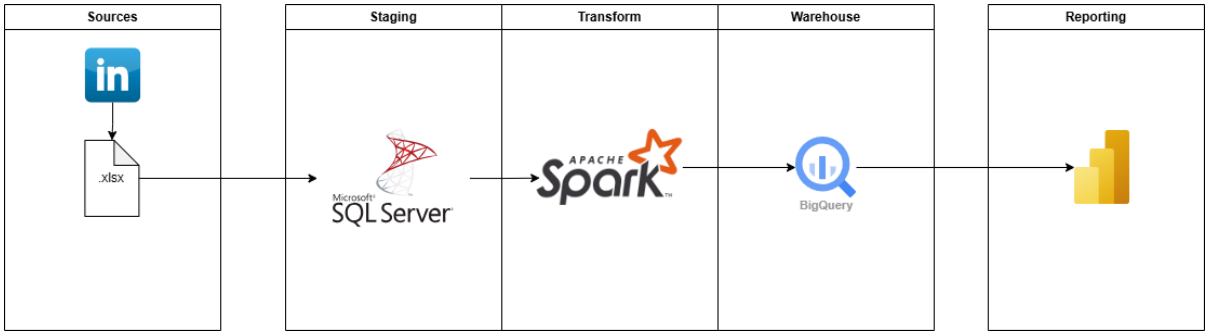
Jupyter Notebook

Microsoft SQL Server

Warehouse

Reporting Insight

2. Pipeline Architecture



This project is an end-to-end data engineering pipeline designed to analyze job market trends from LinkedIn data. The pipeline starts from data sources such as LinkedIn scraping results exported into xlsx files. The raw data is first loaded into Microsoft SQL Server as a staging layer, where it is stored in its original structured format. This staging database acts as a controlled environment for initial storage before further processing.

In the transform layer, Apache Spark is used to clean, standardize, and process the data, including deduplication, timestamp normalization, and feature engineering. The transformed dataset is then loaded into PostgreSQL as the data warehouse layer, structured for analytical queries. Finally, Power BI is used in the reporting layer to create interactive dashboards that visualize job trends, skills demand, and posting patterns, enabling data-driven insights from the processed data.

3. Web Scraping Linkedin Data

Tools *: Jupyter Notebook, Python (BeautifulSoup, request, pandas)*

The web scraping process was built using Python to collect job posting data from LinkedIn based on specific keywords and filters. The script extracts relevant information such as job title, company name, location, posting date, and job

description, then structures the data into a pandas DataFrame (linkedinJob_df). Basic cleaning steps like removing duplicates and handling missing values are applied before exporting the dataset to xlsx, which is then used as the raw input for the staging layer in the data pipeline.

3.1. Import Library (Python)

Code :

```
import requests
from bs4 import BeautifulSoup as bs
import pandas as pd
import random
import time
```

The code imports several libraries required for the web scraping process: *requests* to send HTTP requests and retrieve webpage content, *BeautifulSoup* from *bs4* to parse and extract data from the HTML structure, *pandas* to organize the scraped data into a structured DataFrame, and *random* and *time* to create delays between requests to mimic human browsing behavior.

3.2. Initialization variables

Code :

```
job_title = ""
location = "Indonesia"
geoId = "102478259"
```

initializes three variables: *job_title* (the keyword for job searching), *location* (set to "Indonesia" to filter jobs by country), and *geoId* (LinkedIn's geographic identifier for Indonesia), which are later used to construct the search query URL dynamically.

Code :

```
# Scrape all job IDs across all pages
linkedinJobs_idList = []
start = 0
step = 25
max_pages = 1000  # batas maksimal halaman
```

The list *linkedinJobs_idList* is created to store all collected job IDs. The variable *start* is initialized to 0 and is typically used as an offset parameter in the LinkedIn search URL to paginate through results. The *step* variable is set to 25 because LinkedIn usually displays 25 job postings per page, so the offset increases by 25 for each new page. Finally, *max_pages* is set to 1000 as a safety limit to prevent excessive scraping and control how many pages the script will attempt to access.

3.3. Loop Function

Code:

```
while True:
    linkedin_url = f"https://www.linkedin.com/jobs-
guest/jobs/api/seeMoreJobPostings/search?keywords={job_title}&location=
{location}&geoId={geoId}&f_TPR=r2592000&position=1&pageNum=0&start={sta
rt}"
    response = requests.get(linkedin_url)
    print(f"Scraping start={start} | Status: {response.status_code}")

    if response.status_code != 200 or not response.text.strip():
        print("No more data or error fetching page.")
        break

    soup = bs(response.content, "html.parser")
    linkedinPage_jobs = soup.find_all("li")
    if not linkedinPage_jobs:
        print("No more job listings found.")
        break

    for linkedinJobs in linkedinPage_jobs:
        base_card_div = linkedinJobs.find("div", {"class": "base-
card"})
        if base_card_div and base_card_div.get("data-entity-urn"):
            linkedinJobs_id = base_card_div.get("data-entity-
urn").split(":")[3]
            linkedinJobs_idList.append(linkedinJobs_id)

    start += step
    if start >= max_pages * step:
        print(f"Reached page limit ({max_pages} pages). Stopping
scraping.")
        break

    time.sleep(1)

print(f"Total job IDs collected: {len(linkedinJobs_idList)}")
```

Implements a loop to continuously scrape job IDs from LinkedIn search result pages using pagination. Inside the while True loop, a dynamic URL is constructed using the *job_title*, *location*, *geoId*, and *start* offset parameters. The script sends a request using *requests.get()* and checks the response status. If the response is not successful (status code not 200) or the page is empty, the loop stops. The HTML

content is parsed using *BeautifulSoup*, and all *<li> elements* representing job listings are extracted.

For each listing, the script locates the *div* with *class base-card* and retrieves the data-entity-urn attribute, which contains the unique job ID. The job ID is extracted and appended to *linkedinJobs_idList*. The start value is then incremented by *step (25)* to move to the next page. The loop also includes a safety limit using *max_pages* and a *time.sleep(1)* delay to avoid sending requests too quickly. Finally, the total number of collected job IDs is printed.

3.4. Scrape Job Information Details

Code :

```
# Scrape each job's detail
linkedinJob_list = []
for linkedinJob_id in linkedinJobs_idList:
    linkedin_job_url = f"https://id.linkedin.com/jobs-
guest/jobs/api/jobPosting/{linkedinJob_id}"
    linkedin_job_response = requests.get(linkedin_job_url)

    print(f"Scraping Job ID {linkedinJob_id} | Status:
{linkedin_job_response.status_code}")

    # delay scraping time to avoid rate limit
    time.sleep(random.uniform(3, 7))

    if linkedin_job_response.status_code != 200:
        continue

    linkedin_job_soup = bs(linkedin_job_response.content,
"html.parser")

    linkedinJob_post = {"linkedinJob_id": linkedinJob_id}

    # job title
    linkedinJob_post["linkedinJob_title"] = (
        linkedin_job_soup.find("h2", {"class": "top-card-
layout__title"}).text.strip()
        if linkedin_job_soup.find("h2", {"class": "top-card-
layout__title"})
        else None
    )

    # company name
    linkedinJob_post["linkedinCompany_name"] = (
        linkedin_job_soup.find("a", {"class": "topcard__org-name-link
topcard__flavor--black-link"}).text.strip()
        if linkedin_job_soup.find("a", {"class": "topcard__org-name-
link topcard__flavor--black-link"})
```

```
        else None
    )

    # location
    linkedinJob_post["linkedinJob_location"] = (
        linkedin_job_soup.find("span", {"class": "topcard__flavor
topcard__flavor--bullet"}).text.strip()
        if linkedin_job_soup.find("span", {"class": "topcard__flavor
topcard__flavor--bullet"})
        else None
    )

    # job criteria
    for item in linkedin_job_soup.find_all("li", {"class":
"description__job-criteria-item"}):
        header = item.find("h3", {"class": "description__job-criteria-
subheader"})
        value = item.find("span", {"class": "description__job-criteria-
text description__job-criteria-text--criteria"})

        if header and value:
            header_text = header.get_text(strip=True)
            value_text = value.get_text(strip=True)

            if "Seniority" in header_text or "Tingkat" in header_text:
                linkedinJob_post["linkedinJob_seniority"] = value_text
            elif "Employment" in header_text or "Jenis" in header_text:
                linkedinJob_post["linkedinJob_employmentType"] =
value_text
            elif "Function" in header_text or "Fungsi" in header_text:
                linkedinJob_post["linkedinJob_function"] = value_text
            elif "Industr" in header_text:
                linkedinJob_post["linkedinJob_industries"] = value_text

    # number of applicant
    linkedinJob_post["linkedinNumber_applicants"] = (
        linkedin_job_soup.find ("span", {"class" : "num-
applicants__caption"}).text.strip()
        if linkedin_job_soup.find ("span", {"class" : "num-
applicants__caption"})
        else None
    )


    # posted date
    linkedinJob_post["linkedinJob_postedTime"] = (
        linkedin_job_soup.find("span", {"class": "posted-time-
ago__text"}).text.strip()
```

```
        if linkedin_job_soup.find("span", {"class": "posted-time-
ago__text"})
        else None
    )

    linkedinJob_list.append(linkedinJob_post)
```

This code scrapes detailed information for each collected job ID by iterating through *linkedinJobs_idList*. For each job ID, it constructs a specific job posting URL and sends a request to retrieve the job detail page. A randomized delay *(random.uniform(3, 7))* is added to reduce the risk of rate limiting. If the response is successful (status 200), the HTML content is parsed using *BeautifulSoup*, and key job attributes are extracted, including *job title*, *company name*, *location*, *seniority level*, *employment type*, *job function*, *industry*, *number of applicants*, and *posted time*.

Each extracted value is stored in a dictionary (*linkedinJob_post*) with None assigned if the element is not found, ensuring robustness against missing fields. Finally, each job's structured data is appended to *linkedinJob_list*, creating a complete dataset of detailed job postings ready for conversion into a DataFrame.

3.5. Convert to Dataframe

Code:

```
# Convert to DataFrame
linkedinJob_df = pd.DataFrame(linkedinJob_list)
print(linkedinJob_df.head())
```

This code converts the list of scraped job dictionaries (*linkedinJob_list*) into a structured pandas DataFrame using *pd.DataFrame()*. Each dictionary in the list represents one job posting, and pandas automatically organizes the keys as column names and the values as row entries. The *print(linkedinJob_df.head())* statement displays the first five rows of the DataFrame, allowing you to quickly preview the dataset and verify that the scraping process successfully collected and structured the job data correctly.

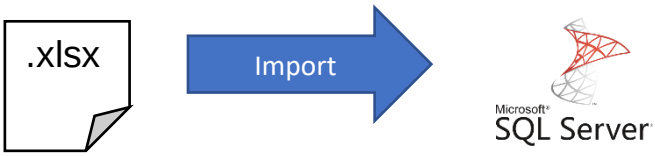
3.6. Export to xlsx

Code:

```
linkedinJob_df.to_excel("linkedin_jobs.xlsx", index=False)
```

After generating the *xlsx* file from the scraping process, download it and upload into Microsoft SQL Server as a staging table. The staging layer is used to store raw structured data before further transformation using Apache Spark.

4. Staging Layer MS SQL Server

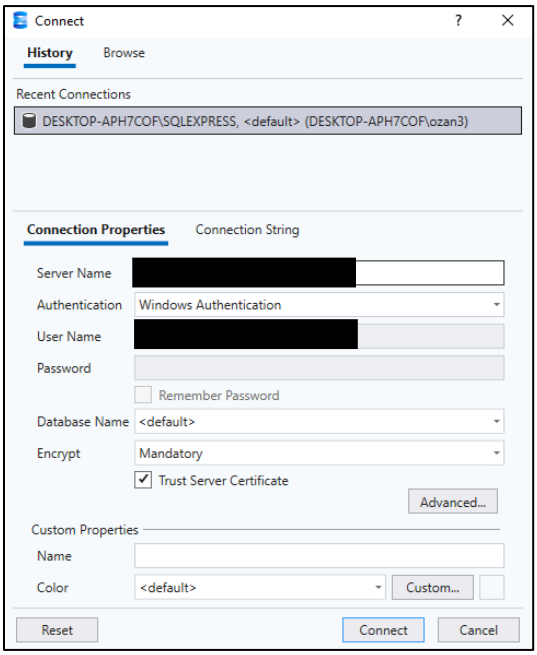


Microsoft SQL Server acts as an intermediate storage layer between the raw data source and the transformation process.

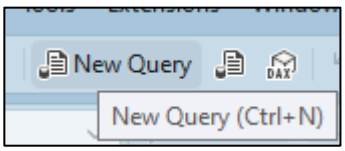
4.1. Import xlsx file to Microsoft SQL Server

This step ensures that the raw job posting data is stored in a structured relational database for further processing and validation. By loading the xlsx into SQL Server, the data becomes easier to manage, query, and integrate with the transformation layer (Apache Spark), while maintaining data integrity before moving into the data warehouse stage.

- Connect to Microsoft SQL Server

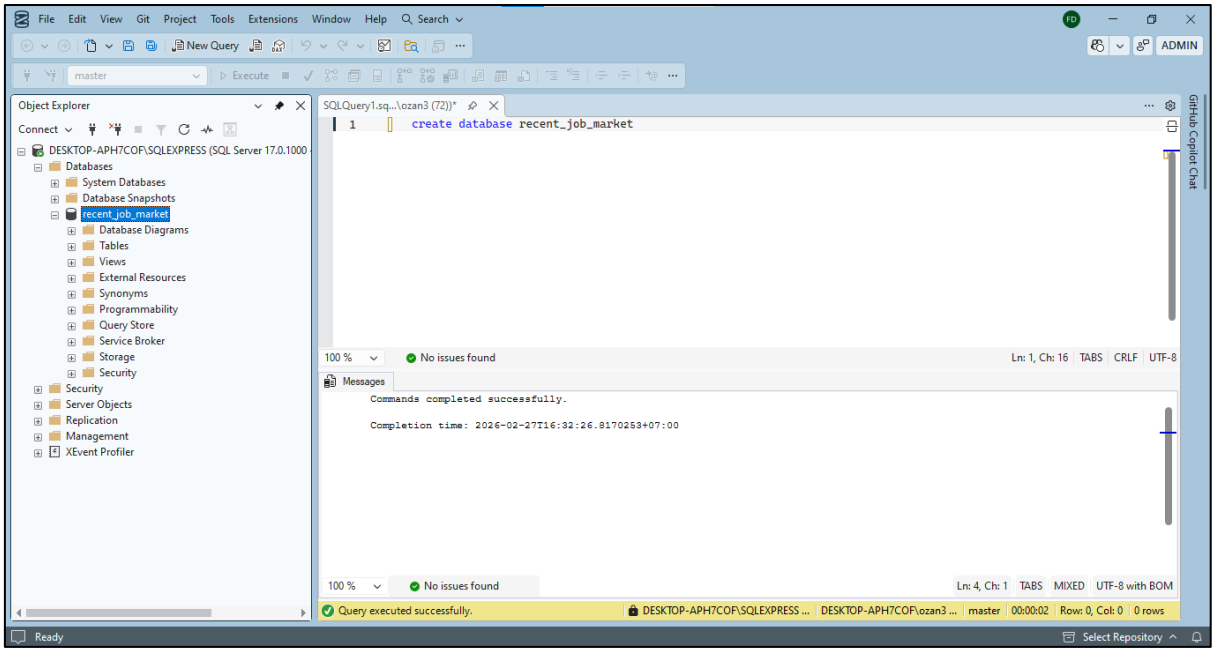


- Create new query

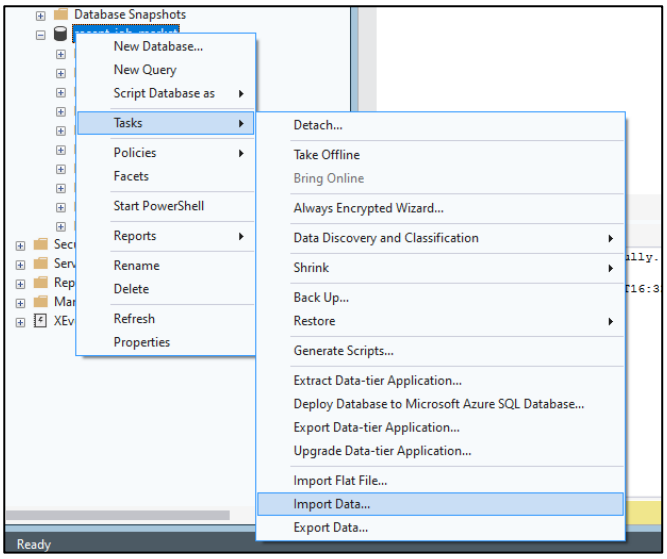


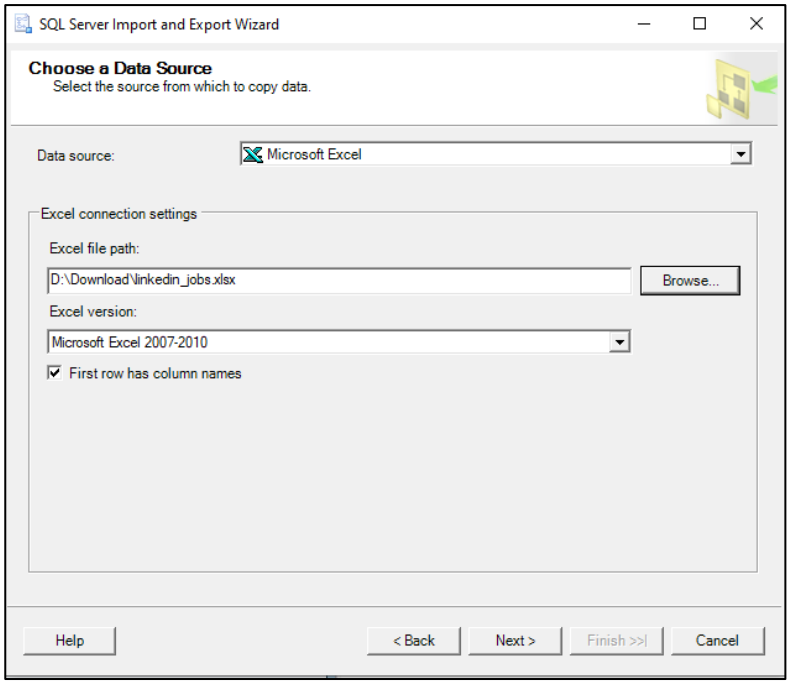
Code :

```
create database recent_job_market
```

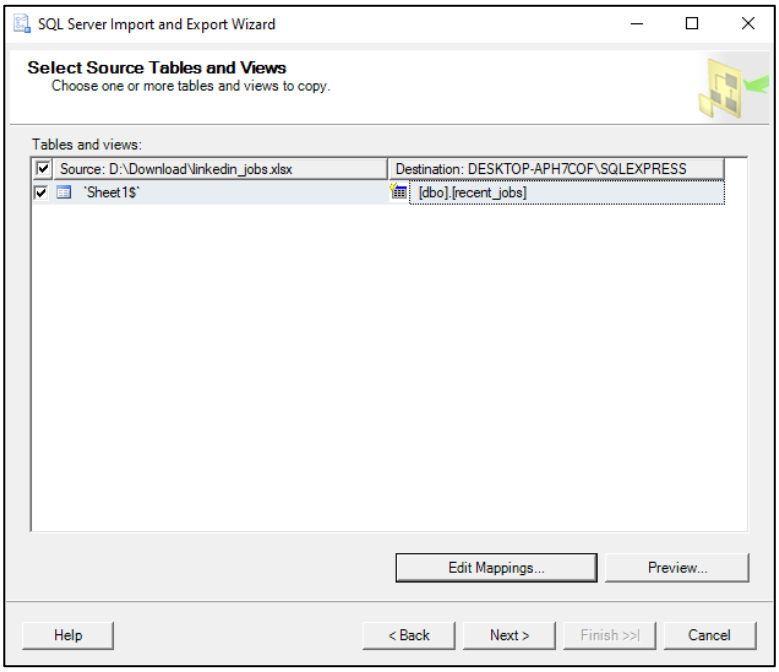


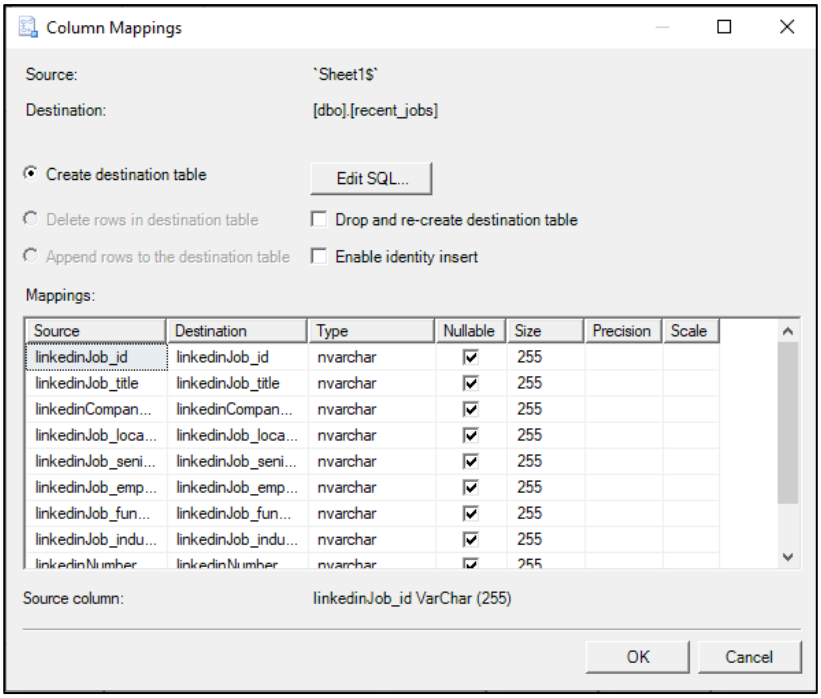
- Search for *linkedin_jobs.xlsx* file





- Check the data





- Test Table

Code :

```
SELECT DB_NAME() AS CurrentDatabase;

USE recent_job_market;
GO

SELECT * FROM dbo.[recent_jobs];
```

Results | Messages

	linkedinJob_id	linkedinJob_title	linkedinCompany_name	linkedinJob_location	linkedinJob_seniority	linkedinJob_
1	4375724428	Finance Accounting Staff	Kalbe Nutritionals (PT Sanghiang Perkasa)	Karawang Barat	Tingkat pemula	Penuh wakt
2	4370258070	Officer Development Program (ODP) Retail Banking	PT. BANK NEGARA INDONESIA (Persero) Tbk.	Jakarta Raya, Indonesia	Tingkat pemula	Penuh wakt
3	4376705498	Staff GA (Pulogadung)	PT. Sarimelati Kencana Tbk.	Jakarta Timur	Tingkat pemula	Kontrak
4	4375151664	Import Staff	PT. Surya TOTO Indonesia Tbk	Area DKI Jakarta	Tidak Berlaku	Penuh wakt
5	4368981113	Future Bankers Development Program	PT Bank Sinarmas Tbk	Jakarta Raya, Indonesia	Tingkat pemula	Penuh wakt
6	4376037380	Real Estate	PT. Sarimelati Kencana Tbk.	Jakarta Raya, Indonesia	Senior tingkat menengah	Penuh wakt
7	4351189305	Business Development Officer	Kalbe Consumer Health	Jakarta Raya, Indonesia	Asosiasi	Penuh wakt
8	4376476960	Social Media Junior Officer	PT Bank Sinarmas Tbk	Jakarta Raya, Indonesia	Tingkat pemula	Kontrak
9	4371106829	Business Finance & Analytics Officer	PT. Sarimelati Kencana Tbk.	Jakarta Raya, Indonesia	Senior tingkat menengah	Penuh wakt
10	4330630847	Frontliner	PT Bank Sinarmas Tbk	Jakarta Raya, Indonesia	Asosiasi	Penuh wakt

5. Apache Spark for Data Processing

In the transform stage, Apache Spark is used to process the staging data stored in Microsoft SQL Server. Spark connects to SQL Server using JDBC and performs data cleaning and transformation operations.

The transformation process includes:

- Removing duplicate job records based on *job_id*
- Handling missing values using *fillna()* and *dropna()*
- Standardizing text fields (lowercase, trimming spaces)
- Casting columns into appropriate data types

Spark was chosen because it is a scalable distributed data processing framework commonly used in modern data engineering pipelines.

5.1. Connect Spark to Microsoft SQL Server & Check The Data

Code:

```
import pandas as pd
from sqlalchemy import create_engine

# Create engine
engine = create_engine(
    "mssql+pyodbc://DESKTOP-APH7COF\\SQLEXPRESS/recent_job_market"
    "?driver=ODBC+Driver+17+for+SQL+Server"
    "&trusted_connection=yes"
)

# Read data
query = "SELECT * FROM dbo.recent_jobs"

pdf = pd.read_sql(query, engine)

print(pdf.head())
```

This code connects to a Microsoft SQL Server database named *recent_job_market* running on the local SQL Server Express instance (**DESKTOP-APH7COF\SQLEXPRESS**) using **SQLAlchemy** with the **ODBC Driver 17 for SQL Server** and Windows trusted authentication. It then executes a SQL query (*SELECT * FROM dbo.recent_jobs*) to retrieve all records from the *recent_jobs* table into a Pandas DataFrame (pdf) using *pd.read_sql()*. Then, it prints the first few rows of the DataFrame to preview the extracted data.

5.2. Convert Data to Spark Dataframe & Check it Again

Code:

```
import os
from pyspark.sql import SparkSession

# Replace this with YOUR python.exe path
python_path = r"D:\python 3.10\python.exe"

os.environ["PYSPARK_PYTHON"] = python_path
os.environ["PYSPARK_DRIVER_PYTHON"] = python_path

spark = SparkSession.builder \
    .appName("ETLPipeline") \
    .master("local[*]") \
    .getOrCreate()

spark_df = spark.createDataFrame(pdf)
```

Configures and initializes a local Apache Spark session using a specific Python 3.10 interpreter located at *D:\python 3.10\python.exe* by setting the **PYSPARK_PYTHON** and **PYSPARK_DRIVER_PYTHON** environment variables to ensure compatibility between Spark and the Python runtime. It then creates a SparkSession named "*ETLPipeline*" running in local mode *("local[*]")*, which uses all available CPU cores. It then converts an existing Pandas DataFrame (pdf) into a Spark DataFrame (spark_df) so the data can be processed using Spark's distributed computing framework.

Code:

```
spark_df.printSchema()

spark_df.show(5)
```

After that, displays the structure of the Spark DataFrame, showing each column name along with its corresponding data type. And then, triggers execution and displays the first five rows of the Spark DataFrame in a tabular format.

```
root
|-- linkedinJob_id: string (nullable = true)
|-- linkedinJob_title: string (nullable = true)
|-- linkedinCompany_name: string (nullable = true)
|-- linkedinJob_location: string (nullable = true)
|-- linkedinJob_seniority: string (nullable = true)
|-- linkedinJob_employmentType: string (nullable = true)
|-- linkedinJob_function: string (nullable = true)
|-- linkedinJob_industries: string (nullable = true)
|-- linkedinNumber_applicants: string (nullable = true)
|-- linkedinJob_postedTime: string (nullable = true)
```

5.3. Remove Duplicate Data

Code:

```
from pyspark.sql.functions import col

spark_df = spark_df.filter(col("linkedinJob_id").isNotNull())
spark_df = spark_df.dropDuplicates(["linkedinJob_id"])
```

Performs basic data cleansing and deduplication on the Spark DataFrame. It filters out any records where *linkedinJob_id* is null to ensure that only valid job entries are retained, since this field acts as the unique identifier. Then, it removes duplicate records based on *linkedinJob_id,* keeping only one occurrence per job posting. Together, these steps ensure the dataset maintains data integrity by excluding invalid entries and preventing duplicate job records from propagating through downstream processing.

5.4. Normalize Timestamp

```
from pyspark.sql import functions as F

spark.conf.set("spark.sql.session.timeZone", "Asia/Jakarta")

spark_df = spark_df \
    .withColumn("amount",
        F.regexp_extract("linkedinJob_postedTime", r"(\d+)",
1).cast("int")
    ) \
    .withColumn("unit",
        F.regexp_extract("linkedinJob_postedTime", r"\d+\s+(\w+)", 1)
    ) \
    .withColumn("seconds_to_subtract",
        F.when(F.col("unit") == "menit", F.col("amount") * 60)
         .when(F.col("unit") == "jam", F.col("amount") * 3600)
         .when(F.col("unit") == "hari", F.col("amount") * 86400)
         .when(F.col("unit") == "minggu", F.col("amount") * 604800)
         .when(F.col("unit") == "bulan", F.col("amount") * 2592000)
         .when(F.col("unit") == "tahun", F.col("amount") * 31536000)
    ) \
    .withColumn(
        "linkedinJob_postedTime",
        (
            F.current_timestamp().cast("long") -
F.col("seconds_to_subtract")
        ).cast("timestamp")
    ) \
    .drop("amount", "unit", "seconds_to_subtract")
```

This transformation converts the relative time stored in `*linkedinJob_postedTime*` (e.g., “2 minggu lalu”) into a proper Spark `timestamp` type. First, it explicitly sets the session timezone to `Asia/Jakarta` to ensure consistent timestamp calculation. It then extracts the numeric value (`*amount*`) and time unit (`*unit*`) from the string using regular expressions. Based on the detected unit (minutes, hours, days, weeks, months, or years), the logic converts the relative time into total seconds (`*seconds_to_subtract*`). Last, it subtracts those seconds from the current timestamp to generate an actual timestamp value, overwriting the original `*linkedinJob_postedTime*` column and dropping the intermediate helper columns to keep the schema clean.

5.5. Change Data Type

Code:

```
#For linkedinNumber_applicants
from pyspark.sql.functions import col, regexp_extract

spark_df = spark_df.withColumn(
    "linkedinNumber_applicants",
    regexp_extract(col("linkedinNumber_applicants"), r"(\d+)",
1).cast("int")
)
```

This code block cleans and transforms the *linkedinNumber_applicants* column, which originally contains text values such as "180 Applicants". Using *regexp_extract*, the code extracts only the numeric portion of the string (e.g., 180) based on the regular expression pattern *(\d+),* which captures consecutive digits. The extracted value is then cast to an integer to ensure the column is stored as a proper numeric metric. This transformation converts semi-structured text data into a structured integer field.

5.6. Normalize linkedinJob_location

This step standardizes raw job location text into structured geographic attributes: city, province, and country.

Code:

```
#Check Data
spark_df.select("linkedinJob_location").show(100, truncate=False)


#Clean the Location Column
from pyspark.sql.functions import col, lower, trim, regexp_replace
```

```
spark_df = spark_df.withColumn(
    "location_clean",
    trim(lower(col("linkedinJob_location")))
)

patterns = [
    ", indonesia",
    " indonesia",
    "area ",
    " dan sekitarnya",
    " raya",
    "kota "
]

for p in patterns:
    spark_df = spark_df.withColumn(
        "location_clean",
        regexp_replace(col("location_clean"), p, "")
    )
```

First, the original *linkedinJob_location* column is inspected to understand format variations. A temporary column *location_clean* is then created by converting text to lowercase, trimming whitespace, and removing inconsistent patterns such as “Indonesia”, “area”, “raya”, “dan sekitarnya”, and “kota”. This ensures uniform formatting before classification.

```
#City
from pyspark.sql.functions import when, initcap

spark_df = spark_df.withColumn(
    "linkedinJob_city",
    when(col("location_clean").contains("jakarta"), "Jakarta")
    .when(col("location_clean").contains("gambir"), "Jakarta")
    .when(col("location_clean").contains("bandung"), "Bandung")
    .when(col("location_clean").contains("surabaya"), "Surabaya")
    .when(col("location_clean").contains("bekasi"), "Bekasi")
    .when(col("location_clean").contains("tangerang selatan"),
"Tangerang Selatan")
    .when(col("location_clean").contains("tangerang"), "Tangerang")
    .when(col("location_clean").contains("karawang"), "Karawang")
    .otherwise(initcap(col("location_clean")))
)
```

Next, rule-based mapping logic is applied to derive *linkedinJob_city* using keyword matching (e.g., “jakarta”, “bandung”, “surabaya”). The detected cities are standardized into proper case formatting to maintain consistency.

```
#Mapping Logic Provience
spark_df = spark_df.withColumn(
    "linkedinJob_province",
    when(col("linkedinJob_city") == "Jakarta", "DKI Jakarta")
    .when(col("linkedinJob_city").isin("Bandung", "Bekasi",
"Karawang"), "Jawa Barat")
    .when(col("linkedinJob_city").isin("Tangerang", "Tangerang
Selatan"), "Banten")
    .when(col("linkedinJob_city") == "Surabaya", "Jawa Timur")
    .otherwise(None)
)
```

After city normalization, province mapping is performed using conditional logic to assign each city to its corresponding province (e.g., Jakarta → DKI Jakarta, Bandung → Jawa Barat). The linkedinJob_country column is then added as a constant value ("Indonesia") since all records fall within the same country.

```
#Country
from pyspark.sql.functions import lit

spark_df = spark_df.withColumn(
    "linkedinJob_country",
    lit("Indonesia")
)
#Drop Column
spark_df = spark_df.drop("linkedinJob_location", "location_clean")
```

Finally, the original raw location column and the intermediate cleaning column are dropped to maintain a clean and structured dataset schema.

5.7. Final Schema

Code: spark_df.printSchema()

The final Spark DataFrame schema contains structured LinkedIn job data with properly defined data types for analysis. Unique job identifiers, textual job attributes (title, company, seniority, employment type, function, industries, city, province, country) are stored as **string**, the number of applicants is stored as **integer**, and the posting time is stored as a **timestamp**. The dataset is now cleanly structured and ready for analytics.

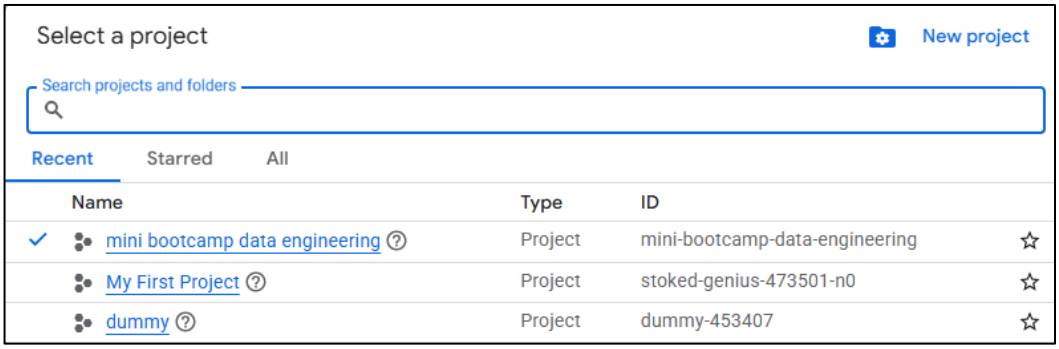
```
root
 |-- linkedinJob_id: string (nullable = true)
 |-- linkedinJob_title: string (nullable = true)
 |-- linkedinCompany_name: string (nullable = true)
 |-- linkedinJob_seniority: string (nullable = true)
 |-- linkedinJob_employmentType: string (nullable = true)
 |-- linkedinJob_function: string (nullable = true)
```

```
|-- linkedinJob_industries: string (nullable = true)
|-- linkedinNumber_applicants: integer (nullable = true)
|-- linkedinJob_postedTime: timestamp (nullable = true)
|-- linkedinJob_city: string (nullable = true)
|-- linkedinJob_province: string (nullable = true)
|-- linkedinJob_country: string (nullable = false)
```

6. Data Warehousing

A data warehouse using Bigquery as a centralized and structured storage layer where cleaned and transformed data from Spark is permanently stored for analytical purposes. In this pipeline, Bigquery holds normalized LinkedIn job data in relational tables with well-defined schemas, enabling efficient SQL-based querying, aggregation, and indexing.

6.1. Set Up Bigquery



Steps:

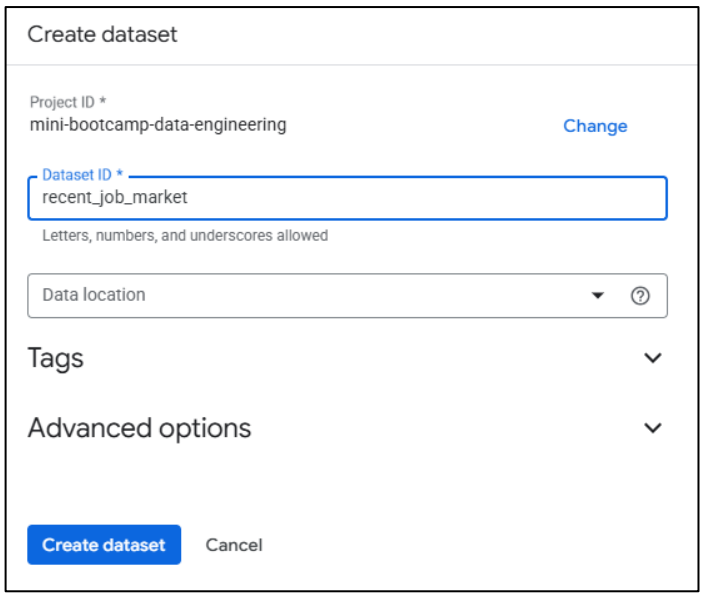
- Open the BigQuery Console in Google Cloud.
- Click the Select a project dropdown.
- Select the project to use.

In this first image, the selected project is:

- Project Name: mini bootcamp data engineering
- Project ID: mini-bootcamp-data-engineering

Technical Explanation:

- The project ID is globally unique and will be used in connection configurations
- All datasets and tables will be located under this project.
- Ensure the selected project is connected to an active billing account.

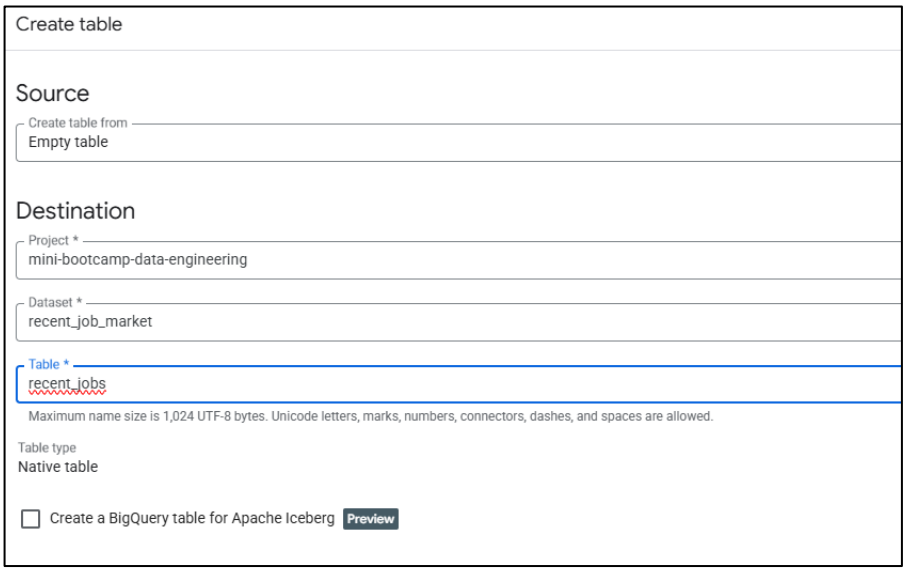


In the second image, a dataset is created with the following configuration:

- Project ID: mini-bootcamp-data-engineering
- Dataset ID: recent_job_market

Steps:

- Click Create Dataset.
- Enter Dataset ID: recent_job_market.
- Click Create dataset.



In the third image, the configuration used is:

- Source: Empty table
- Project: mini-bootcamp-data-engineering

- Dataset: recent_job_market
- Table name: recent_jobs
- Table type: Native table

Steps:

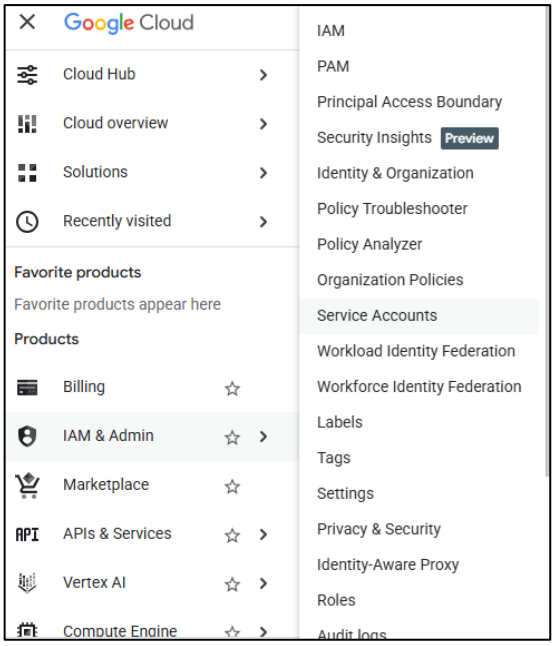
- Click Create Table.
- Select Source: Empty table

Make sure the destination is:

- Dataset: recent_job_market
- Table: recent_jobs
- Select Native table (default).
- Click Create Table.

6.2. Create Service Account

This section describes how to create a service account that will be used by Spark to write data into BigQuery within the mini-bootcamp-data-engineering project.



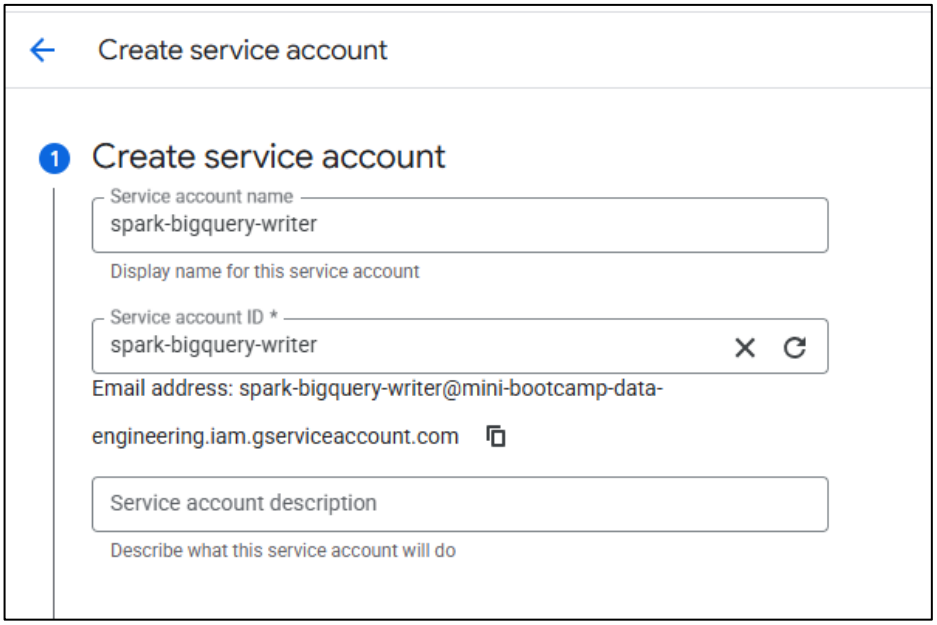
Navigate to Service Accounts Menu

In the first image, the Google Cloud Console navigation menu is opened.

Steps:

- Go to Google Cloud Console.
- Click IAM & Admin.
- Select Service Accounts.

This menu allows you to create and manage service accounts that applications (such as Spark jobs) will use for authentication and authorization.



Create Service Account (Basic Configuration)

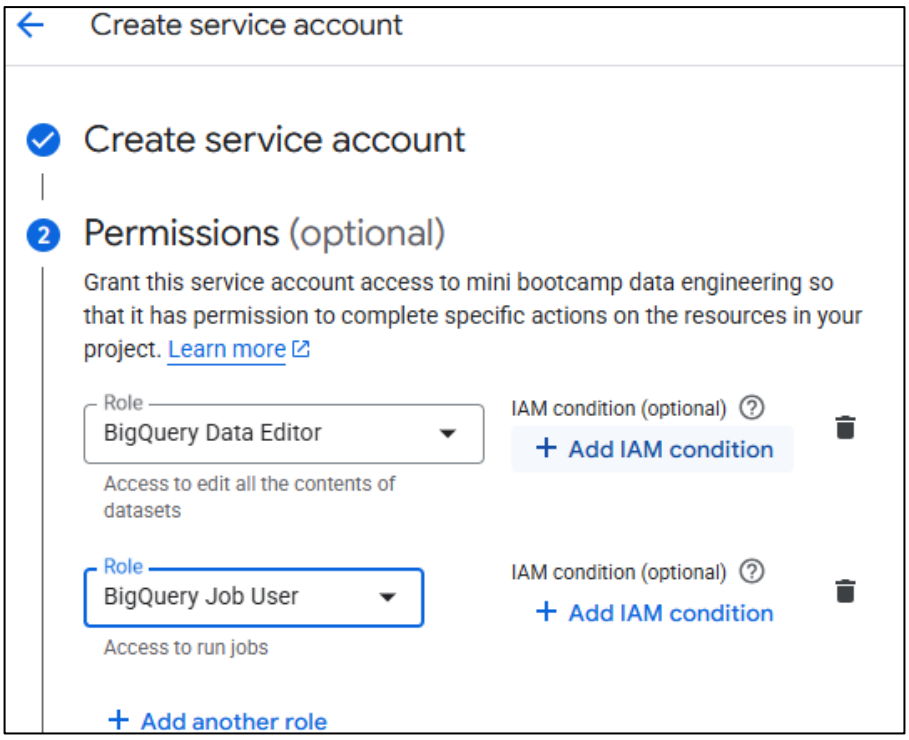
In the second image, a new service account is being created with the following configuration:

- Service account name: spark-bigquery-writer
- Service account ID: spark-bigquery-writer
- Email address:
- spark-bigquery-writer@mini-bootcamp-data-engineering.iam.gserviceaccount.com

Steps:

- Click Create Service Account.
- Enter:
- Service account name: spark-bigquery-writer
- Click Create and Continue.

This service account will be used by Spark jobs to authenticate securely to Google Cloud.



Assign IAM Roles (Permissions)

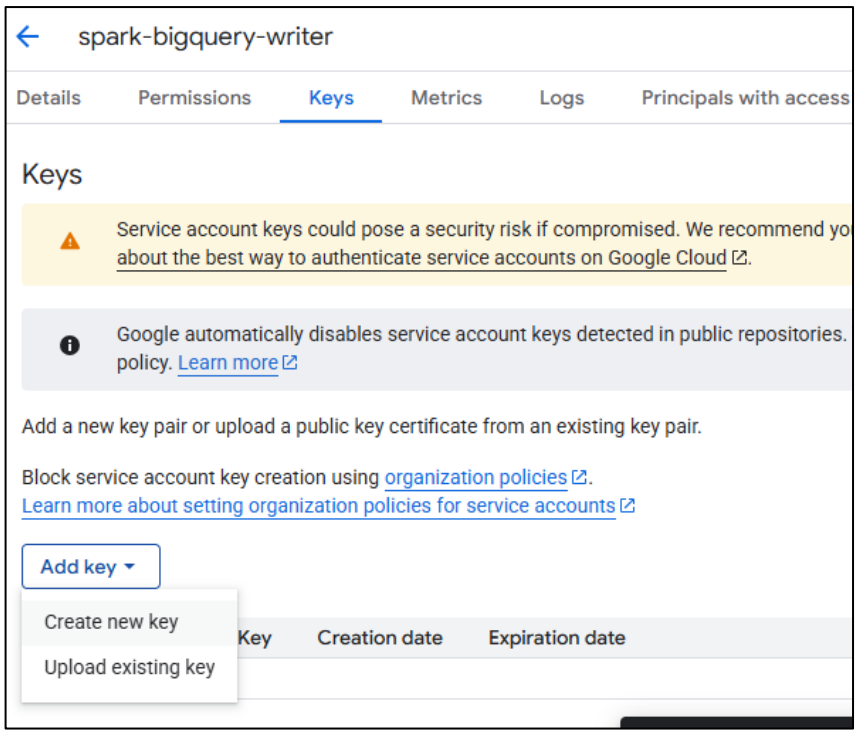
In the third image, IAM roles are assigned to the service account.

Roles granted:

- BigQuery Data Editor : Allows editing dataset contents (create/update tables, insert data).
- BigQuery Job User : Allows running BigQuery jobs.

Steps:

- Under Permissions, click Select a role.
- BigQuery Data Editor
- BigQuery Job User
- Click Continue.
- Click Done.



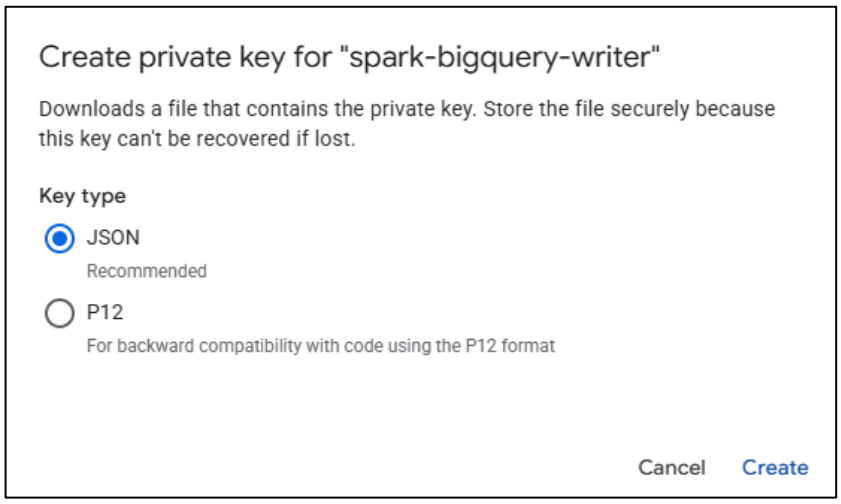
Create Service Account Key

In the fourth image, the Keys tab of the service account is opened.

Steps:

- Open the created service account: spark-bigquery-writer.
- Navigate to the Keys tab.
- Click Add Key.
- Select Create new key.

A key is required when authenticating from external environments



Generate JSON Private Key

In the fifth image, the private key creation dialog appears.

Configuration:

- Key type: JSON (Recommended)

Steps:

- Select JSON.
- Click Create.

Download and securely store the generated .json file.

⚠ Important Security Notes:

- The JSON key file contains private credentials.
- Store it securely (never commit to Git repositories).
- Rotate or revoke the key if compromised.
- Prefer Workload Identity when running inside GCP environments.

6.3. Convert spark_df into csv.file

Code:

```
pandas_df = spark_df.toPandas()

pandas_df.to_csv("linkedin_jobs_cleaned.csv", index=False)
```

In this step, I converted the processed Spark DataFrame (*spark_df*) into a Pandas DataFrame using *.toPandas()*. After that, I exported the Pandas DataFrame into a CSV file named *linkedin_jobs_cleaned.csv*.

For this project, I used the simplest approach by converting *spark_df* into a CSV file before loading it into BigQuery. This method is suitable for small-scale datasets and makes the data loading process easier to manage in a local development environment.

6.4. Load to Bigquery

Code:

```
from google.cloud import bigquery
from google.oauth2 import service_account

credentials = service_account.Credentials.from_service_account_file(
    "mini-bootcamp-data-engineering-f6b829fb22d4.json"
)

client = bigquery.Client(credentials=credentials)

table_id = "mini-bootcamp-data-
engineering.recent_job_market.recent_jobs"
```

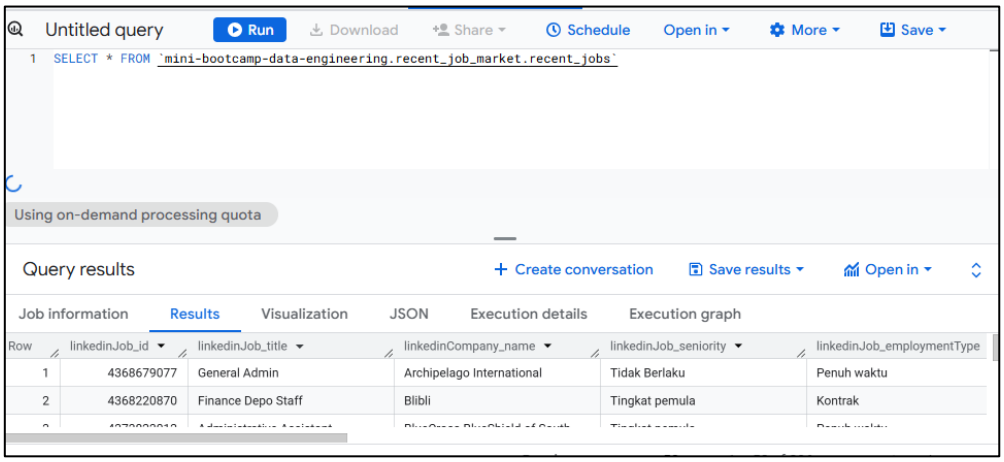
```
job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    autodetect=True,
)

with open("linkedin_jobs_cleaned.csv", "rb") as source_file:
    job = client.load_table_from_file(
        source_file,
        table_id,
        job_config=job_config
    )

job.result()

print(" CSV successfully loaded to BigQuery.")
```

In this step, I used the Google Cloud BigQuery Python client library to load the processed data from a CSV file into a BigQuery table. First, I authenticated to Google Cloud using a service account JSON key to securely establish a connection. Then, I initialized a BigQuery client and defined the destination table within the project and dataset. I configured a *LoadJobConfig* to specify that the source format is CSV, skip the header row, and automatically detect the table schema. Using *load_table_from_file()*, I uploaded the *linkedin_jobs_cleaned.csv* file to BigQuery, and *job.result()* was executed to ensure the loading process completed successfully before confirming that the data was successfully stored in the data warehouse.

6.5. Final Check data on Bigquery

Code:

```
SELECT * FROM `mini-bootcamp-data-engineering.recent_job_market.recent_jobs`
```



6.6. Transform data for *linkedinJob_id* & *linkedinNumber_applicants*

Code:

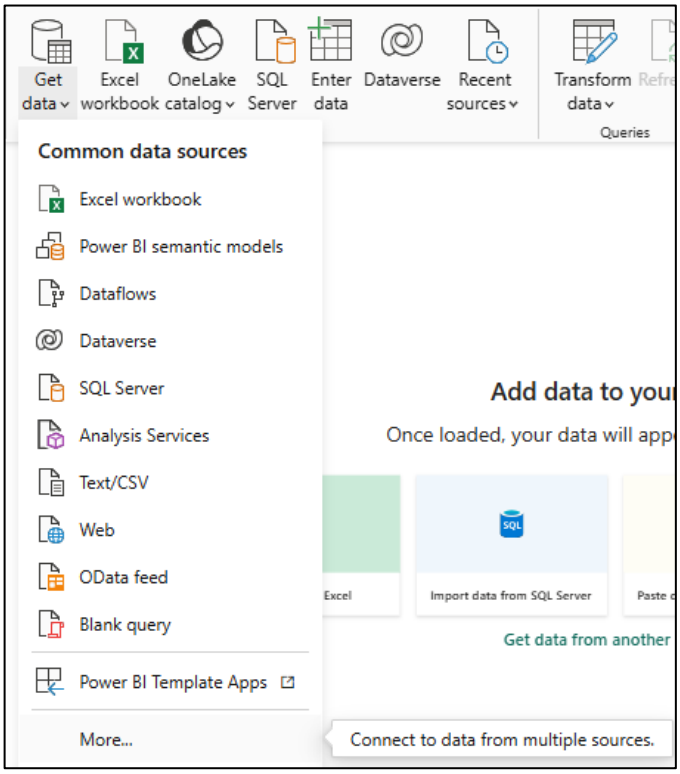
```
CREATE OR REPLACE TABLE recent_job_market.recent_jobs AS
SELECT
    CAST(linkedinJob_id AS STRING) AS linkedinJob_id,
    linkedinJob_title,
    linkedinCompany_name,
    linkedinJob_seniority,
    linkedinJob_employmentType,
    linkedinJob_postedTime,
    linkedinJob_city,
    linkedinJob_country,
    linkedinJob_province,
    CAST(linkedinNumber_applicants AS INT64) AS linkedinNumber_applicants
FROM recent_job_market.recent_jobs;
```

In this step, the data types of *linkedinJob_id* and *linkedinNumber_applicants* are standardized in **Google BigQuery** to prevent misconfiguration issues in **Microsoft Power BI**. The *linkedinJob_id* column is converted from INTEGER to STRING to ensure it is treated strictly as an identifier (not aggregated or summed in reports), while *linkedinNumber_applicants* is converted from FLOAT to INT64 to remove decimal precision and ensure accurate counting and aggregation.

7. Reporting Insight

We use Microsoft Power BI to transform warehouse data into interactive dashboards and actionable insights quickly and efficiently. It integrates well with cloud data sources like BigQuery, supports scalable data models, and enables stakeholders to explore KPIs and trends in a secure, shareable environment while keeping heavy processing in the data warehouse.

7.1. Set up data on PowerBI



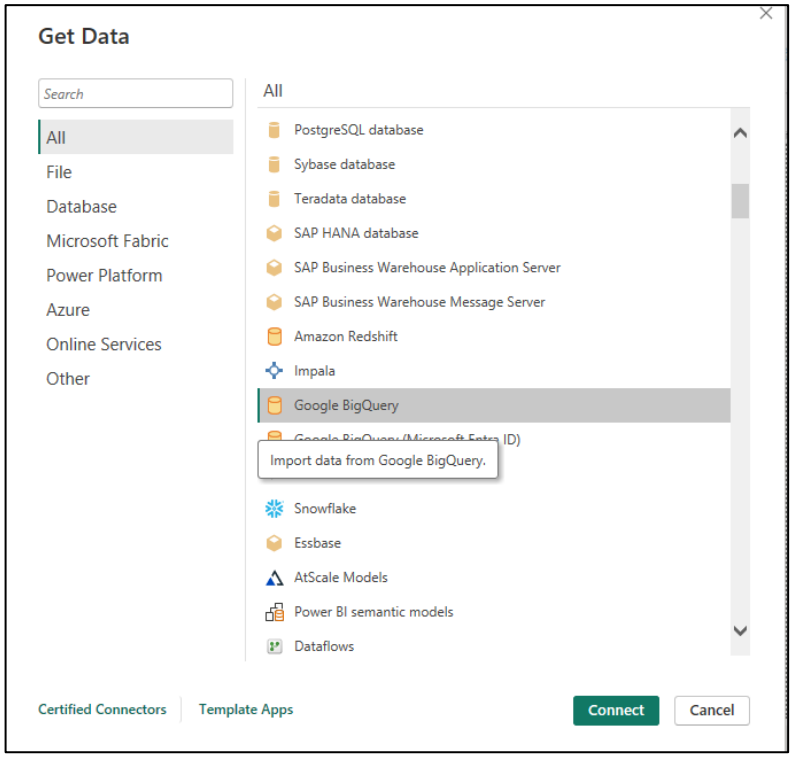
Open Get Data Menu

In the first image, the Get Data dropdown menu in Power BI Desktop is opened.

Steps:

- Launch Power BI Desktop.
- Navigate to the Home tab.
- Click Get Data.
- Select More… to open the full connector list.

This step allows you to choose from all available data connectors supported by Power BI.



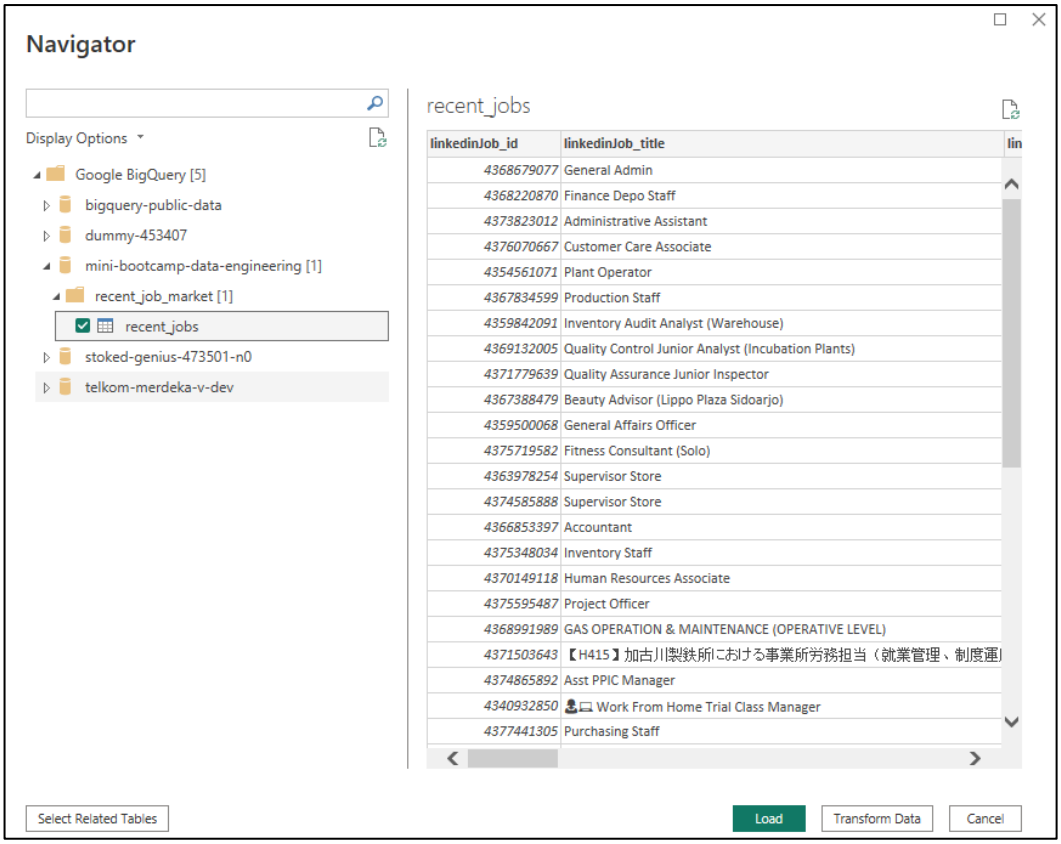
Select Google BigQuery Connector

In the second image, the Get Data window is displayed with the connector list.

Steps:

- In the left panel, ensure All is selected.
- Scroll down through the available database connectors.
- Select Google BigQuery.
- Click Connect.

Power BI uses a native BigQuery connector that supports OAuth authentication and direct integration with Google Cloud projects. **Sign in with your gcp account after.**



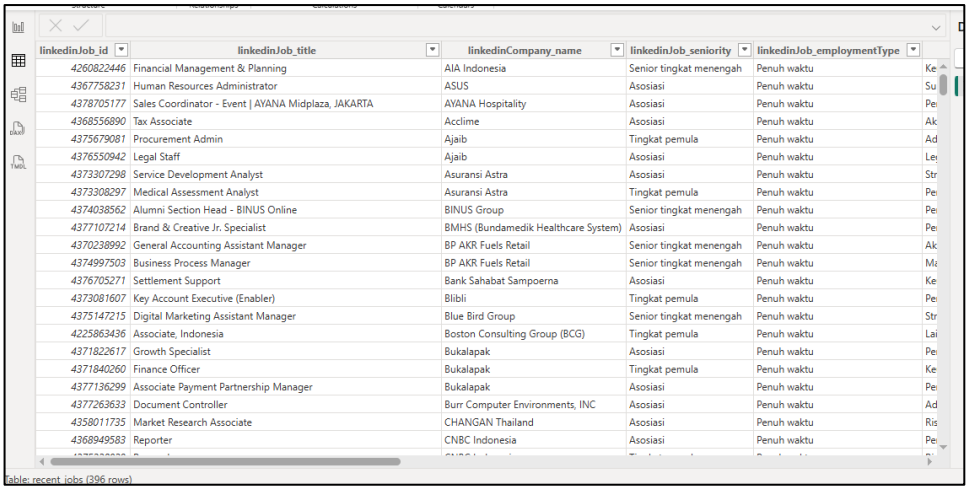
After clicking Connect, Power BI will prompt authentication.

Steps:

- Choose Sign in.
- Log in using your Google account.
- Authorize Power BI to access BigQuery resources.
- Confirm successful authentication.

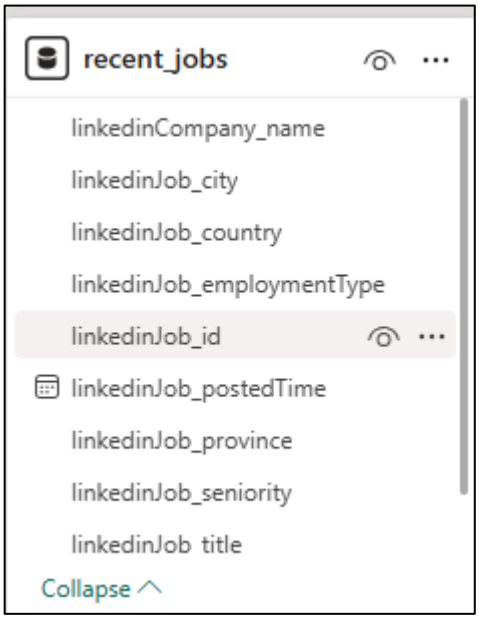
7.2. Check the Data

After successfully loading the recent_jobs table from BigQuery into Power BI, perform the following validation steps to ensure data integrity, structure correctness, and readiness for reporting



linkedinJob_id	linkedinJob_title	linkedinCompany_name	linkedinJob_seniority	linkedinJob_employmentType
4260822446	Financial Management & Planning	AIA Indonesia	Senior tingkat menengah	Penuh waktu
4367758231	Human Resources Administrator	ASUS	Asosiasi	Penuh waktu
4378705177	Sales Coordinator - Event | AYANA Midplaza, JAKARTA	AYANA Hospitality	Asosiasi	Penuh waktu
4368556890	Tax Associate	Acclime	Asosiasi	Penuh waktu
4375679081	Procurement Admin	Ajaib	Tingkat pemula	Penuh waktu
4376550942	Legal Staff	Ajaib	Asosiasi	Penuh waktu
4373307298	Service Development Analyst	Asuransi Astra	Asosiasi	Penuh waktu
4373308297	Medical Assessment Analyst	Asuransi Astra	Tingkat pemula	Penuh waktu
4374038562	Alumni Section Head - BINUS Online	BINUS Group	Senior tingkat menengah	Penuh waktu
4377107214	Brand & Creative Jr. Specialist	BMHS (Bundamedik Healthcare System)	Asosiasi	Penuh waktu
4370238992	General Accounting Assistant Manager	BP AKR Fuels Retail	Senior tingkat menengah	Penuh waktu
4374997503	Business Process Manager	BP AKR Fuels Retail	Senior tingkat menengah	Penuh waktu
4376705271	Settlement Support	Bank Sahabat Sampoerna	Asosiasi	Penuh waktu
4373081607	Key Account Executive (Enabler)	Blibli	Tingkat pemula	Penuh waktu
4375147215	Digital Marketing Assistant Manager	Blue Bird Group	Senior tingkat menengah	Penuh waktu
4225863436	Associate, Indonesia	Boston Consulting Group (BCG)	Tingkat pemula	Penuh waktu
4371822617	Growth Specialist	Bukalapak	Asosiasi	Penuh waktu
4371840260	Finance Officer	Bukalapak	Tingkat pemula	Penuh waktu
4377136299	Associate Payment Partnership Manager	Bukalapak	Asosiasi	Penuh waktu
4377263633	Document Controller	Burr Computer Environments, INC	Asosiasi	Penuh waktu
4358011735	Market Research Associate	CHANGAN Thailand	Asosiasi	Penuh waktu
4368949583	Reporter	CNBC Indonesia	Asosiasi	Penuh waktu

Table: recent_jobs (396 rows)



Validate Table Preview (Data View)

From the image, the Data View of Power BI displays the recent_jobs table with columns such as:

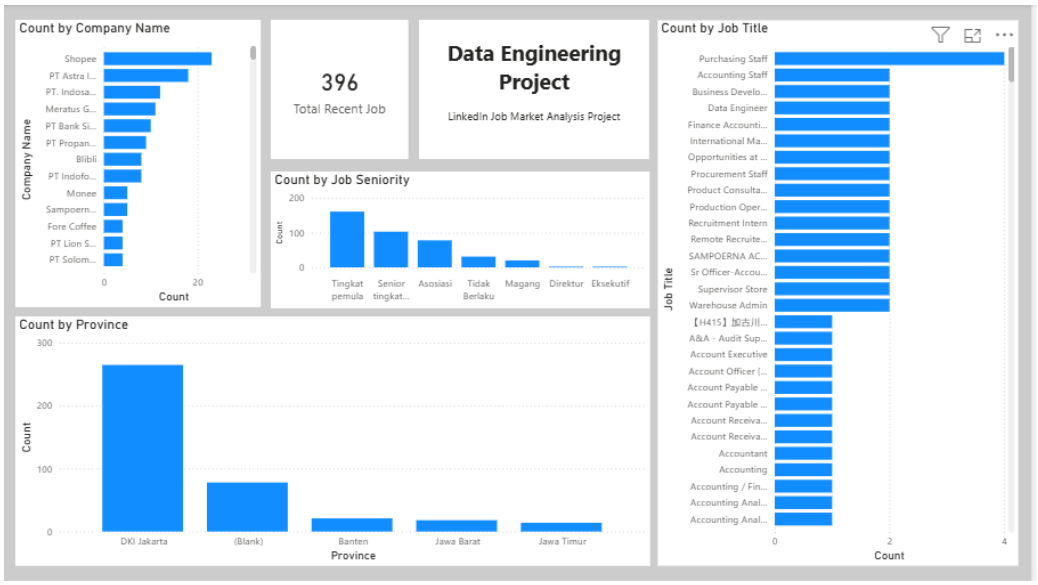
- linkedinJob_id
- linkedinJob_title
- linkedinCompany_name
- linkedinJob_seniority
- linkedinJob_employmentType

Validation Checklist:

✔ Check Row Count

- Confirm total row count (bottom left status bar shows: 396 rows).
- Cross-check with BigQuery

7.3. Create Graph



The process of creating graphs using Power BI to visualize data in a more informative and understandable form. The style and type of visualization can be adjusted to suit the needs of the analysis, but in this project the visuals used include Cards to display total data and Clustered Bar Charts to show comparisons of numbers based on categories such as company name, position, seniority level, and province. This section also explains the purpose of each visual, the stages of its creation, and the role of graphs in supporting the process of data analysis and interpretation.

8. Conclusion

- **What are the job trends in the last 30 days?**

Over the past 30 days, **Purchasing Staff** positions have emerged as the most frequently posted roles. This indicates strong demand in procurement and supply chain-related functions. The trend suggests that companies are actively strengthening their operational and inventory management capabilities, possibly due to business expansion or supply chain optimization efforts. The consistency of postings in this role reflects stable hiring demand within operational departments.

- **Which companies are hiring the most in the last 30 days?**

Based on the data, **Shoppee** shows the highest number of job postings within the last 30 days. This suggests aggressive hiring activity, potentially driven by business growth, market expansion, or workforce scaling initiatives. A high posting volume typically indicates strategic recruitment efforts, particularly in competitive industries such as e-commerce and digital retail.

- **What is the employment type trend?**

The dominant employment level trend is **Entry Level**. This indicates that organizations are primarily recruiting early-career talent. Such a trend may reflect workforce expansion strategies, cost optimization, or a focus on building a pipeline of junior professionals who can be developed internally. It also suggests increased opportunities for fresh graduates and individuals at the beginning of their careers.

9. Additional

Full Project link :

https://github.com/ozandx/End-to-end-Job-Market-Trends-Pipeline.git